

# Smyst

## Produkt- & Feature-Konzept

Eine produktgetriebene Sicht auf den aktuellen Code-Stand der Smyst-App: Welche Features sind heute gebaut, wie hängen sie zusammen, was liefert wirklich Nutzerwert — und welche Lücken müssen vor dem öffentlichen Launch geschlossen werden.

|             |                                                              |
|-------------|--------------------------------------------------------------|
| Erstellt am | 28. April 2026                                               |
| Version     | 1.0 · Parallel zum bestehenden Strategie-Konzept             |
| Code-Basis  | smyst-app v0.1.0 (Vite + React 18 + TypeScript + Tailwind 3) |
| Umfang      | 5 Views · 4-Step Builder · Persona Engine · Memory-Pipeline  |
| Sprache     | Deutsch                                                      |
| Zielgruppe  | Produkt, Engineering, Stakeholder                            |

*„Dein Leben. Deine Stimme. Für immer.“*

Hinweis: Dieses Dokument ist eine eigenständige Ergänzung zum bereits vorhandenen Smyst\_Konzept (Strategie, GTM, Monetarisierung). Es betrachtet das Produkt aus der Feature-Perspektive — was gebaut ist, wie es sich anfühlt, was funktional fehlt.

# Inhaltsverzeichnis

|    |                                 |    |
|----|---------------------------------|----|
| 1  | Executive Summary               | 3  |
| 2  | Aktueller Produktstand          | 4  |
| 3  | Informationsarchitektur         | 5  |
| 4  | Kern-Features im Detail         | 6  |
| 5  | User Journeys & Flows           | 10 |
| 6  | Datenmodell & Persona Engine    | 12 |
| 7  | Funktionale Lücken              | 14 |
| 8  | Produkt-Roadmap (Feature-Sicht) | 15 |
| 9  | Design & Tonalität              | 16 |
| 10 | Tech-Snapshot                   | 17 |

# Executive Summary

Smyst ist eine Digital-Twin-Plattform, die persönliche Identität — Werte, Erinnerungen, Sprachstil, Entscheidungsmuster — in einen konversationsfähigen digitalen Zwilling überführt. Der aktuelle Code-Stand ist ein vollständiger UI-Prototyp: Landing Page, vierstufiger Twin Builder, Memory-Upload-Oberfläche, Twin-Chat und Dashboard sind vorhanden und navigierbar, jedoch noch ohne echtes Backend, Auth oder AI-Integration.

Das Produkt positioniert sich primär B2C (Familien-Legacy, persönliche Weisheit) mit klaren B2B-Andockpunkten (Founder-Legacy, Expertenwissen). Diese Doppelpositionierung ist im UI an mehreren Stellen explizit (Use-Cases-Sektion, B2B-Kontakt-CTA, „Auch für Unternehmenswissen geeignet“).

## Kerngedanke in einem Satz

*„Smyst ist nicht das Archiv eines Lebens, sondern dessen fortgeführte Stimme.“*

## Wo das Produkt heute steht

- **UI-Prototyp komplett** — alle fünf Hauptansichten sind in `App.tsx` implementiert (~874 Zeilen, single-file React).
- **Visuelle Sprache definiert** — Glassmorphism, Mesh-Gradient, Manrope/Space-Grotesk-Typografie, Farbsystem Deep Blue / Violet / Cyan.
- **Inhalte vorhanden** — Use-Cases, Feature-Beschreibungen, Trust-Elemente und Tonalität sind bereits getextet und konsistent.
- **Was fehlt** — Backend, Auth, AI-Integration, persistente Daten, Audio-/Video-Verarbeitung, Routing (statt useState-View-Switching), Tests, i18n.

# Aktueller Produktstand

Der Code beschreibt ein klar abgegrenztes Produkt mit fünf Ansichten, die über einen `useState`-basierten View-Switch verbunden sind. Es gibt noch kein React-Router-Setup, obwohl `react-router-dom` bereits in den Dependencies steht — das ist der erste konkrete Hinweis auf geplante Erweiterung.

## Bestehende Views

| View          | Funktionsumfang heute                                                                                                                               |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Landing Page  | Hero, Vision, 6 Use-Cases, 4 Produktbereiche, Sicherheit, CTA. Produktmarketing-Seite mit B2C-Schwerpunkt und B2B-Andockpunkten.                    |
| Twin Builder  | 4-Schritte-Wizard mit Progress Bar: (1) Stammdaten, (2) Werte & Überzeugungen, (3) Persönlichkeit & Stil, (4) Zugriff & Legacy. Endet im Dashboard. |
| Memory Upload | Drag-and-Drop-Zone, Upload-Liste mit Verarbeitungsstatus, Memory-Kategorien (Kindheit, Beruf, Familie, ...) und Statistiken.                        |
| Twin Chat     | Chat-Interface mit zufälligen Mock-Antworten, vorgefertigten Vorschlägen, Antwort-Stil-Auswahl und Quellen-Toggle.                                  |
| Dashboard     | Begrüßung, drei Aktions-Karten (Chat, Memory, Settings), Aktivitätsübersicht und Twin-Status mit drei Health-Metriken.                              |

## Was bereits gut sitzt

- **Konsistente Tonalität** — ruhig, vertrauenswürdig, nicht-technisch. Mockup-Beispiele wie „Empathisch, reflektiert, ruhig entscheidend“ treffen das Markenversprechen ohne Pathos.
- **Ehrlicher Umgang mit AI** — explizit als „Modell-Ausgabe gekennzeichnet“, Quellen-Anzeige im Chat-Sidebar als Toggle vorgesehen.
- **Privacy-First-Signale** — DSGVO-Badge, Server in Deutschland, Ende-zu-Ende-Verschlüsselung, ISO 27001 sind in der Security-Sektion verankert (noch als Versprechen, nicht als Implementierung).

# Informationsarchitektur

Die App teilt sich in einen **Marketing-Modus** (Landing Page) und einen **Produkt-Modus** (Dashboard, Builder, Memory, Chat). Der Wechsel passiert über den Header-Button „Early Access“ / „Zurück zur Landing Page“ — eine Lösung, die für den Prototyp elegant ist, aber für die Produktion durch Auth + Routing ersetzt werden muss.

## Navigationsstruktur

| Bereich                 | Items                                                                           |
|-------------------------|---------------------------------------------------------------------------------|
| Marketing-Header        | Vision · Anwendungen · Produkt · Sicherheit · i@smyst.com · <b>Early Access</b> |
| Produkt-Header          | Dashboard · Twin Builder · Memory · Chat · <b>Zurück zur Landing Page</b>       |
| Footer (gleichbleibend) | Produkt-Links · Unternehmen · Rechtliches · LinkedIn / Instagram / Mail         |

## Empfehlung

Der duale Header signalisiert klar zwei Modi und ist intuitiv. Sobald Auth dazukommt, sollte der Wechsel jedoch nicht mehr ein manueller Button sein, sondern automatisch per Login-Status. Die Landing Page bleibt öffentlich, das Produkt liegt hinter /app/\*-Routes mit react-router-dom (bereits installiert, noch nicht integriert).

## Kern-Features im Detail

Vier Produktbereiche tragen das Smyst-Versprechen: Twin Builder, Memory Engine, Persona Engine und Twin Chat. Drei sind heute als UI vollständig sichtbar; die vierte (Persona Engine) ist konzeptionell verankert, aber erwartungsgemäß noch nicht implementiert.

### FEATURE 4.1

#### Twin Builder — der erste 15-Minuten-Twin

Vierstufiger Wizard, der die Persona-Grundlage erfasst: **Schritt 1** Stammdaten (Name, Alter, Beruf), **Schritt 2** Werte und weiterzugebende Lebensweisheit, **Schritt 3** Kommunikationsstil (Dropdown: warm, direkt, humorvoll, weise) plus Entscheidungsmuster, **Schritt 4** Zugriffsrechte (Familie, Geschäftspartner, öffentlich). Endet im Dashboard. Progress-Anzeige in 25%-Schritten.

#### Stärke

Der Builder ist bewusst kurz. Der Versprechenstext „In 15 Minuten zum ersten Twin“ ist mit nur vier Schritten realistisch — das senkt die Aktivierungshürde erheblich.

#### Risiko

Vier Schritte erzeugen einen Twin, der noch oberflächlich ist. Es braucht ein klares *Progressive-Profiling*-Modell: Der erste Twin ist eine Skizze, der „echte“ Twin entsteht durch wiederholte Memory-Uploads und Chat-Konversationen über Wochen.

### FEATURE 4.2

#### Memory Upload — Erinnerungen werden Wissen

Drag-and-Drop-Zone akzeptiert PDF, DOC, TXT, MP3, WAV, JPG, PNG, MP4 (max. 100 MB pro Datei). Status-Pipeline: *Wird verarbeitet* → *Verarbeitet*. Sidebar zeigt Memory-Kategorien (Kindheit, Beruf, Familie, Reisen, Werte) und Statistiken (Gesamt-Memories, verarbeitete Dateien, Speichernutzung).

#### Stärke

Die Pipeline-Metapher (Upload → Verarbeitung → Kategorie) macht für den Nutzer fühlbar, dass etwas Sinnvolles mit den Dateien passiert. Das ist wichtig, weil die eigentliche Magie (Embedding, semantische Suche) für den Nutzer unsichtbar ist.

#### Risiko

Audio-/Video-Transkription ist im UI versprochen, aber technisch anspruchsvoll: Sprecher-Identifikation, Dialekte, lange Aufnahmen, Hintergrundgeräusche. Hier sollte die V1.0 ehrlich starten — Text und PDF first, Audio in V1.5.

#### FEATURE 4.3

### Persona Engine — der Stil hinter den Antworten

Im Code nicht direkt sichtbar (kein Backend), aber im Marketing klar positioniert: Die Engine verbindet Profil (aus dem Builder), Memories (aus den Uploads), Sprachstil und Werte zu einem Kontext, der jede Chat-Antwort vor der Generierung „färbt“. Antworten sollen „glaubwürdig, ruhig und persönlich“ klingen — der Anti-Generic-Bot.

#### Stärke

Die Trennung Persona-Engine ↔ LLM-Backend ist die richtige Architektur: Sie erlaubt einen Wechsel des Sprachmodells (OpenAI → Anthropic → lokales Modell) ohne dass die Twin-Identität verloren geht.

#### Risiko

Im Demo-Code liefert der Twin Chat zufällige Antworten aus einem vierelementigen Array. Der Sprung zu echter Persona-Treue ist der härteste technische Schritt im gesamten Produkt — und gleichzeitig das Kernversprechen. Hier muss am ehrlichsten erwartet werden, dass die ersten Versionen *noch nicht* klingen wie der echte Mensch.

#### FEATURE 4.4

### Twin Chat — Gespräche mit Kontext und Haltung

Klassisches Chat-Interface mit Header (Name + „Online · Persönlicher Twin“), Nachrichten-Stream und Eingabefeld. Sidebar bietet vier Vorschläge („Was ist dein wichtigster Lebensrat?“ ...) und drei Einstellungen: Antwort-Stil, Kontext-Nutzung, Quellen-Anzeige.

#### Stärke

Die Vorschläge im Sidebar sind ein cleveres Cold-Start-Hilfsmittel: Der Nutzer weiß bei einem Twin selten sofort, was er fragen soll — vier kuratierte Beispielfragen geben den Einstieg.

#### Risiko

Es fehlt heute die *Nachvollziehbarkeit*: Der Sidebar-Toggle „Quellen anzeigen“ existiert, aber es gibt im Mock keine Anzeige, *welche* Erinnerung zur Antwort geführt hat. Genau diese Verknüpfung ist das, was Smyst von einem generischen LLM-Chatbot unterscheidet — sie muss prominent gebaut werden.

# User Journeys & Flows

Aus dem Code lassen sich drei prototypische Journeys ableiten — alle drei sind im UI heute bereits begehbar.

## Journey A — „Die Bewahrerin“ (B2C, 50–65)

**Auslöser:** Anzeige auf Instagram, Headline „Deine Enkel sollen dich kennenlernen“. Klick auf Anzeige → Landing Page.

|    |                                 |                                                                |
|----|---------------------------------|----------------------------------------------------------------|
| 1. | <b>Landing Page</b>             | Liest Hero, scrollt zu Use-Cases. „Großeltern → Enkel“ trifft. |
| 2. | <b>CTA „Starte deinen Twin“</b> | Wechselt in Twin Builder.                                      |
| 3. | <b>Builder Schritt 1–4</b>      | Gibt Stammdaten, Werte, Stil und Zugriff (Familie ✓) ein.      |
| 4. | <b>Dashboard</b>                | „Willkommen zurück, Anna“ — Profil 86%, Memory 92%.            |
| 5. | <b>Memory Upload</b>            | Lädt Familienfotos und Sprachnachrichten hoch.                 |
| 6. | <b>Twin Chat</b>                | Probiert Vorschlag „Was bedeutet Familie für dich?“.           |

## Journey B — „Der Nachfolger“ (B2B, 40–50)

**Auslöser:** Empfehlung im Founder-Netzwerk, gezielte Suche nach „Founder Legacy“. Klick auf B2B-Mailto-Link im Footer.

Die heutige App zeigt B2B-Use-Cases prominent, leitet aber Anfragen noch über `b2b@smyst.com` — also klassischer Sales-Prozess. **Empfehlung:** Für die nächste Iteration eine eigene `/business-Landing` mit Demo-Request-Formular und Enterprise-FAQ.

## Journey C — „Die Wiederkehr“ (Engagement)

**Auslöser:** E-Mail-Hinweis „Du hast 18 neue Memories diesen Monat gesammelt“ oder kalendarischer Trigger („Heute vor 2 Jahren ...“).

- Login → **Dashboard** mit Aktivitätsübersicht (Letzte Konversation, neue Memories).
- Klick „Fortsetzen“ → **Twin Chat** mit dem letzten Kontext.
- Twin schlägt vor: „Möchtest du die Geschichte über deinen Vater weiterführen?“ — *im Code heute noch nicht abgebildet, aber klare nächste Iteration.*



# Datenmodell & Persona Engine

Aus den Builder-Schritten und der Memory-Pipeline lässt sich ein minimal-tragfähiges Datenmodell ableiten. Es ist im Code bisher nicht persistent — sobald ein Backend dazukommt, sollte sich die API-Struktur an dieser Logik orientieren.

## Entitäten

| Entität             | Felder & Bedeutung                                                                                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User</b>         | Account-Inhaber. <i>Felder:</i> id, email, password_hash, created_at, locale, plan (free/premium/family/business).                                                                       |
| <b>Twin</b>         | Persona, gehört zu einem User. <i>Felder:</i> id, owner_id, name, age, profession, communication_style, decision_pattern, values_text, legacy_text, completeness_score.                  |
| <b>Memory</b>       | Hochgeladenes Asset, gehört zu einem Twin. <i>Felder:</i> id, twin_id, file_path, mime_type, category, original_filename, uploaded_at, processing_status, transcript, embeddings_vector. |
| <b>Conversation</b> | Chat-Session, gehört zu einem Twin. <i>Felder:</i> id, twin_id, started_at, last_message_at, message_count.                                                                              |
| <b>Message</b>      | Einzelne Nachricht. <i>Felder:</i> id, conversation_id, role (user/twin), content, sources (array of memory_ids), created_at.                                                            |
| <b>AccessGrant</b>  | Wer darf den Twin sehen? <i>Felder:</i> id, twin_id, grantee_email, scope (chat_only/full), active_from, active_until, granted_by.                                                       |

## Persona Engine — Pipeline-Vorschlag

Der Weg von einer User-Frage zur Twin-Antwort sollte in drei expliziten Stufen passieren — das macht die Engine debugbar und erlaubt später die Anzeige von Quellen im Chat:

- **Retrieval** — Frage → Embedding → Top-k semantisch ähnliche Memories (z.B. k=6).
- **Persona Composition** — Twin-Profil (Werte, Stil, Entscheidungsmuster) + Retrieval-Snippets → System-Prompt für das LLM.
- **Generation + Provenance** — LLM-Antwort + Liste der genutzten Memory-IDs → wird im UI als „Quellen“ darstellbar.

## Schnittstelle zum Chat-UI

Die heutige Chat-Komponente hat bereits den Toggle „Quellen anzeigen“ im Sidebar. Das Datenmodell oben unterstützt das ohne weitere Anpassungen — `Message.sources` ist die einzige Verbindung, die das UI braucht.

# Funktionale Lücken

Das, was im Marketing versprochen wird, und das, was im Code heute lebt, fallen an mehreren Stellen auseinander. Diese Lücken sind normal für einen UI-Prototyp — sie sollten aber bewusst priorisiert werden.

| Lücke                         | Beschreibung & Priorität                                                                                                                                                       |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Auth & Persistenz             | Kein Login, kein User-Account, kein Backend. <code>useState</code> verliert alles beim Refresh. <b>Priorität: kritisch.</b>                                                    |
| Routing                       | <code>react-router-dom</code> ist installiert, aber Views werden über einen <code>currentView</code> -State umgeschaltet. <b>Priorität: hoch</b> (auch für Sharing/Deeplinks). |
| Echte AI-Integration          | Twin-Antworten sind ein Mock-Array mit vier Stichpunkten. Echte Persona-Engine fehlt. <b>Priorität: kritisch.</b>                                                              |
| Memory-Verarbeitung           | Upload-UI vorhanden, aber kein Embedding, keine semantische Suche, keine Audio-Transkription. <b>Priorität: hoch.</b>                                                          |
| Quellen-Anzeige im Chat       | Toggle existiert, Anzeige nicht. <b>Priorität: mittel</b> — aber differenzierungsentscheidend.                                                                                 |
| Legacy-Access (Zeitsteuerung) | Builder-Schritt 4 erfasst Zugriffsgruppen, aber „nach meinem Tod“ und „aktiv ab Datum X“ fehlen. <b>Priorität: mittel.</b>                                                     |
| Mobile-Experience             | Tailwind-Layout ist responsive, aber keine native App, kein Voice-Input für Memories. <b>Priorität: mittel.</b>                                                                |
| i18n                          | Texte sind hart auf Deutsch verdrahtet. <b>Priorität: niedrig</b> (DACH-Launch ist Phase 1).                                                                                   |
| Tests                         | Keine Test-Konfiguration im Repo. <b>Priorität: mittel</b> sobald Logik (Persona Engine, Datei-Pipeline) entsteht.                                                             |
| Onboarding-Telemetrie         | Keine Analytics, keine Funnel-Messung. Für die KPIs aus dem Strategie-Konzept (Activation, Retention, Conversion) zwingend nötig. <b>Priorität: mittel.</b>                    |

# Produkt-Roadmap (Feature-Sicht)

Die folgende Roadmap fokussiert ausschließlich Features — also was der Nutzer sehen und tun kann. Sie ergänzt die technische Roadmap des bestehenden Strategiekonzepts.

## Phase 1 · MVP (heute → 3 Monate)

Ziel: Ein einzelner Twin, der textbasiert glaubwürdig antwortet.

- Auth (E-Mail + Magic Link) und persistente Profile
- React-Router-Setup, Deeplinks zu allen Views
- Echte LLM-Anbindung (1 Provider, z.B. Anthropic)
- Memory Upload für Text und PDF mit Embeddings
- Quellen-Anzeige im Chat („Diese Antwort basiert auf ...“)
- Onboarding-Funnel mit Analytics-Events

## Phase 2 · V1.5 (Monat 4–6)

Ziel: Twin wird vielstimmig — Audio kommt dazu.

- Audio-Upload (MP3/WAV) mit Transkription
- Sprecher-Tagging (wer spricht in dieser Aufnahme?)
- Voice-First-Memory-Recording in Mobile (PWA)
- Erweiterter Builder: optionale Tiefenfragen pro Lebensphase
- Legacy-Access mit Datum-/Trigger-Steuerung
- Family-Plan: bis zu 5 Twyns unter einem Account

## Phase 3 · V2.0 (Monat 7–12)

Ziel: B2B-fähig, multimodal, skalierbar.

- Video-Upload mit Sprecher- und Szenen-Erkennung
- Enterprise-Dashboard (mehrere Twyns, Admin-Rechte, Audit-Log)
- API für Drittanbieter (Bestattungsunternehmen, Erbrechts-Plattformen)
- Multi-Language Support (EN zuerst)
- Twin-Embeds (z.B. Bewerbungs-Twin auf eigener Website)
- Kollaboratives Memory-Sammeln („Lade Familie ein, Erinnerungen beizutragen“)

# Design & Tonalität

Das visuelle System ist im Code sehr präzise definiert und sollte konsequent durchgehalten werden, auch in Mobile-, B2B- und Email-Touchpoints.

## Design-Tokens (aus `index.css` und `tailwind.config`)

| Token             | Wert                                                                  |
|-------------------|-----------------------------------------------------------------------|
| Farbe Deep Blue   | #112041 — Primärtext, Status-Akzente                                  |
| Farbe Soft Violet | #8B7CFF — B2B-Tag, Persona-Akzente                                    |
| Farbe Signal Cyan | #59C7FF — Primary-CTA, Progress, B2C-Tag                              |
| Hintergrund       | Mesh-Gradient aus #f5f6f8 → #cfd3d9, vier radiale Hotspots            |
| Glasmorphismus    | Backdrop-Blur 18–32px, weiße Transparenz 12–28%, Inset-Highlight oben |
| Typografie        | Manrope (Body) · Space Grotesk (Headings) — ruhig, hochwertig, lesbar |
| Border-Radius     | 0.75rem Standard, bis 30px für Hero-Karten                            |

## Tonalität — was funktioniert, was nicht

- **Funktioniert:** Kurze, würdevolle Sätze. „Bewahre, was zählt.“ „Dein Leben. Deine Stimme. Für immer.“ — emotional, nicht kitschig.
- **Funktioniert:** Sachlich-warme Twin-Beispielantwort: „Prüfe erst, ob die Entscheidung zu deinen Werten passt.“ — zeigt das Markenversprechen ohne es auszubuchstabieren.
- **Vorsichtig:** Emoji-Use im Dashboard und Use-Cases (■, ■■■■■■, ■ ...). Auf einer Premium-Plattform für ein sensibles Thema kann das billig wirken — Alternative: monochrome Lucide-Icons (lucide-react ist bereits installiert).
- **Vorsichtig:** Health-Scores im Dashboard (86%, 92%, 88%). Gamification eines Lebensbild ist heikel — die Metaphern „Knowledge Health“ und „Gesprächsqualität“ sollten ehrlicher umformuliert werden.

# Tech-Snapshot

Kurze Standortbestimmung der Codebasis — als Anhang für Engineering-Stakeholder.

| Bereich    | Stand                                                                                                         |
|------------|---------------------------------------------------------------------------------------------------------------|
| Framework  | React 18.3.1 + Vite 6.0.5 + TypeScript ~5.6.2                                                                 |
| Styling    | Tailwind CSS 3.4.17, Custom Design Tokens, tailwindcss-animate                                                |
| UI-Lib     | Eigene shadcn-Style-Komponenten (Card, Button) in src/components/ui/                                          |
| Icons      | lucide-react 0.468.0 (installiert, im Code noch nicht genutzt — stattdessen Emoji)                            |
| Routing    | react-router-dom 7.1.1 (installiert, noch nicht eingebunden)                                                  |
| Class-Mgmt | class-variance-authority, clsx, tailwind-merge                                                                |
| Struktur   | Single-File App.tsx mit fünf Komponenten — sauber für Prototyp, sollte bei V1 in src/views/ aufgeteilt werden |
| Build      | npm run dev, npm run build, npm run preview                                                                   |
| Was fehlt  | Backend, Tests, CI/CD, .env-Handling, ESLint-Setup, Storybook                                                 |

Dieses Konzept ist eine Momentaufnahme zum 28. April 2026 und spiegelt den Code-Stand der smyst-app v0.1.0 wider. Die separate Bewertung („Meinung zum Konzept“) ergänzt dieses Dokument um eine kritische Außensicht.